

Technical Documentation Assistant

Implementation Report

RAG-Based Q&A System for Technical Standards

Built with Python, OpenAI, ChromaDB, LangChain, MCP, Streamlit

Portfolio Project for Software Engineering Interview Preparation

April 2026

Table of Contents

1. Project Overview & Architecture
2. Configuration Management
3. PDF Ingestion Pipeline
4. Header/Footer Cleaning
5. Text Chunking Strategy
6. Embedding & Vector Storage
7. Answer Generation (RAG Core)
8. MCP Server (Claude Desktop Integration)
9. Streamlit Web Interface
10. Testing Strategy
11. Evaluation Framework (RAGAS)
12. Containerization (Docker)
13. CI/CD Pipeline
14. Technologies Summary
15. Interview Talking Points

1. Project Overview & Architecture

The Technical Documentation Assistant is a production-structured RAG (Retrieval-Augmented Generation) system that enables natural language Q&A over technical PDF documents. Users upload PDFs, the system ingests, chunks, and embeds them into a vector database, then answers questions with cited, grounded responses.

Core Pipeline Flow

```

PDF Document
  | extract_text_from_pdf()
  v
DocumentContent (structured pages)
  | clean_document()
  v
Cleaned Document (headers/footers removed)
  | chunk_document()
  v
List[Chunk] (overlapping text segments)
  | embed_chunks()
  v
ChromaDB Vector Store (persistent vectors)
  | query_collection() + generate_answer()
  v
Cited Answer with Sources

```

Three Access Interfaces

- Streamlit Web UI - document upload, real-time Q&A with source display
- MCP Server - Claude Desktop integration via stdio transport (3 tools: search, ask, status)
- Direct Pipeline - scripts for testing and batch processing

Key Architectural Decisions

- ChromaDB for vector storage: local persistence, zero infra, easily swappable
- Recursive character splitting: preserves semantic boundaries over naive fixed-size cuts
- Content-based header/footer detection: frequency analysis, works on any PDF layout
- Temperature=0 for LLM calls: RAG answers must be deterministic and factual
- Pydantic models at all boundaries: runtime type validation prevents silent failures
- Dual LLM support (OpenAI + Anthropic): abstraction layer makes providers swappable

2. Configuration Management

All settings are centralized in `src/config.py` using Pydantic Settings. This provides type-validated configuration with a clear priority hierarchy: environment variables > `.env` file > defaults. No hardcoded secrets or magic strings anywhere in the codebase.

Configuration Fields

```
class Settings(BaseSettings):
    # Paths (OS-agnostic via pathlib.Path)
    project_root, data_raw_dir, vectorstore_dir

    # LLM Configuration
    openai_api_key: str          # From .env
    anthropic_api_key: str       # From .env
    llm_provider: str            # 'openai' or 'anthropic'
    llm_model: str               # default: 'gpt-4o-mini'

    # Embedding
    embedding_model: str         # 'text-embedding-3-small'

    # Chunking
    chunk_size: int = 1000      # Target chars per chunk
    chunk_overlap: int = 200    # Sliding window overlap

    # Retrieval
    retrieval_top_k: int = 5    # Results per query
```

Design Pattern: Singleton

A single global 'settings' instance is created on module import and shared across the entire application. This ensures consistent configuration without passing config objects through every function call.

3. PDF Ingestion Pipeline

The ingestion module (`src/ingestion/`) extracts text from PDF documents using PyMuPDF (fitz), a C-based library that is 10-50x faster than pure-Python alternatives like PyPDF2.

Data Models (`src/ingestion/models.py`)

- PageContent: Represents one page (`page_number + text`). Page numbers enable citations.
- DocumentContent: Full document with metadata (`filename, filepath, total_pages, pages list`). Provides `full_text` property for concatenation and `get_page_text()` for safe page access.

Extraction Process (`src/ingestion/pdf_reader.py`)

- Opens PDF with context manager (RAII pattern - resource cleanup guaranteed)
- Iterates pages with 1-indexed page numbers for human-readable citations
- Applies `_clean_page_text()` to each page: collapses excessive blank lines, fixes hyphenation (`auto-\nmatic -> automatic`), strips whitespace
- Skips empty pages to avoid wasting downstream processing on noise
- Error handling: `FileNotFoundError` for missing files, `ValueError` for non-PDFs

Batch Processing

`extract_all_pdfs(directory)` processes every PDF in a directory with per-file error handling. A single corrupt PDF won't crash the entire batch - it logs the error and continues.

4. Header/Footer Cleaning

Technical PDFs like MISRA standards have repeated headers and footers on every page. These pollute retrieved context and waste tokens. The cleaner module (`src/chunking/cleaner.py`) uses a content-based frequency analysis approach that works on any PDF layout.

Detection Algorithm

- 1. Scan all pages and count unique lines per page
- 2. For each unique line, count how many pages it appears on
- 3. If a line appears on $\geq$ threshold% of pages AND is < 200 chars, flag it
- 4. Remove all flagged lines from all pages

Why Content-Based Over Coordinate-Based?

- Works on any PDF layout (header position varies between documents)
- Self-adapting - learns from the document itself, no manual configuration
- Handles multi-line headers naturally
- 200-char limit ensures long content lines are never mistakenly removed

Threshold Parameter (default: 0.4 = 40%)

Controls aggressiveness of detection. Lower threshold = more aggressive removal. Higher = more conservative. A line appearing on 40%+ of pages is almost certainly a header/footer, not meaningful content.

Immutability Pattern

`clean_document()` returns a NEW DocumentContent object - it never mutates the input. This functional approach makes the pipeline easier to reason about and debug, since each step produces a fresh output without side effects.

5. Text Chunking Strategy

Documents can be hundreds of pages, but LLMs have limited context windows. The chunker (`src/chunking/chunker.py`) splits documents into overlapping segments that preserve semantic coherence using LangChain's `RecursiveCharacterTextSplitter`.

Recursive Character Splitting

The splitter tries separators in order of preference:

```
separators = ["\n\n", "\n", ". ", " ", ""]
```

1. Paragraph breaks (`\n\n`) - most natural split point
2. Single newlines (`\n`) - good fallback
3. Sentence boundaries (`.`) - preserve sentences
4. Word boundaries () - last resort
5. Characters () - only for single words `> chunk_size`

Why Not Fixed-Size Splitting?

Fixed-size cuts mid-sentence, destroying context. Example: 'Rule 8.4 states that a compatible decla-' (cut mid-word). Recursive splitting prefers natural boundaries, producing chunks that are semantically coherent.

Configuration

```
chunk_size: int = 1000    # Target characters per chunk
chunk_overlap: int = 200  # Overlap between consecutive chunks
min_chunk_size: int = 100 # Discard chunks smaller than this
```

Overlap (200 chars) ensures that if context spans a chunk boundary, both adjacent chunks contain the relevant passage. `min_chunk_size` filters out noise like stray headers or lone page numbers that survived cleaning.

Metadata Tracking

Each Chunk carries full provenance: `source_filename`, `page_number`, `chunk_index`, `start_char`, `end_char`. This enables precise citations and debugging. The citation property (e.g., 'MISRA-2020.pdf, page 14') is computed automatically.

6. Embedding & Vector Storage

The embedding module (`src/embedding/vector_store.py`) converts text chunks into high-dimensional vectors and stores them in ChromaDB for similarity search.

Embedding Model: OpenAI text-embedding-3-small

- Converts text into 1536-dimensional vectors
- Same model used for both chunks and queries (vector space consistency)
- Cost: ~\$0.02 per 1M tokens (very cheap)

Vector Database: ChromaDB

- Local persistent storage (`vectorstore/` directory) - survives process restarts
- Cosine similarity for nearest-neighbor search
- Each chunk stored with metadata: `source_filename`, `page_number`, `citation`, `chunk_index`
- Idempotent upsert: re-running `embed_chunks` doesn't duplicate vectors

Key Functions

- `get_chroma_client()` - creates or connects to persistent ChromaDB instance
- `get_or_create_collection(client)` - gets/creates 'tech_docs' collection with OpenAI embeddings
- `embed_chunks(chunks)` - batch upsert in groups of 100 for efficiency
- `query_collection(query, n_results=5)` - returns top-k most similar chunks by cosine similarity

Abstraction Layer Design

All vector DB logic is isolated in one module. To swap ChromaDB for Pinecone, Weaviate, or pgvector, you only need to reimplement these 4 functions. The rest of the pipeline doesn't know or care which vector database is being used.

7. Answer Generation (RAG Core)

The generation module (`src/generation/`) is where retrieval meets generation. It takes a user question, retrieves relevant chunks, and uses an LLM to synthesize a cited answer grounded exclusively in the retrieved context.

The RAG Pipeline (`generate_answer`)

```
def generate_answer(question, n_results=5):
    # 1. RETRIEVE: find relevant chunks
    raw_chunks = query_collection(question, n_results)

    # 2. BUILD PROMPT: format context + question
    user_prompt = build_user_prompt(question, raw_chunks)

    # 3. CALL LLM: generate answer
    answer_text = _call_llm(user_prompt)

    # 4. PACKAGE: return Answer object with metadata
    return Answer(question, answer_text, sources, has_answer)
```

Prompt Engineering

The system prompt enforces strict constraints to prevent hallucination:

- Rule 1: Use ONLY information from provided context (no general knowledge)
- Rule 2: After each claim, cite source: [Source: filename, page X]
- Rule 3: If context insufficient, say 'documentation does not contain information'
- Rule 4: Be concise and technical
- Temperature=0: deterministic responses - same question always gives same answer

Multi-Provider LLM Support

The generator supports both OpenAI and Anthropic through an abstraction layer. The `_call_llm()` function dispatches to `_call_openai()` or `_call_anthropic()` based on `settings.llm_provider`. Switching providers requires only a config change, no code changes.

Hallucination Prevention (5 Layers)

- 1. System prompt explicitly forbids using knowledge outside context
- 2. Only retrieved chunks are fed to the LLM (not the full document)
- 3. Citation requirement forces verifiable claims
- 4. `has_answer` flag: honest 'I don't know' beats confident hallucination
- 5. Evaluation metrics (faithfulness) detect hallucinations quantitatively

8. MCP Server (Claude Desktop Integration)

The MCP server (`src/mcp_server/server.py`) exposes the RAG pipeline as standardized tools that any MCP-compatible client can call. It uses FastMCP with stdio transport - Claude Desktop spawns this script as a child process and communicates over stdin/stdout.

Three Exposed Tools

1. `docs_status` - Check Index Status

Returns whether documents are loaded and how many chunks are indexed. Agents should call this first to verify readiness before querying.

2. `docs_search` - Raw Retrieval (No LLM)

Returns raw passages with citations and relevance scores. No LLM synthesis. Use case: agents that want to inspect source material directly or feed passages into their own reasoning chain.

3. `docs_ask` - Full RAG Q&A

Runs the complete RAG pipeline and returns a cited answer. Use case: agents/humans wanting a finished, grounded answer.

Input Validation

All tool inputs are validated with Pydantic models before execution. `SearchInput` requires query (3-500 chars) and `n_results` (1-10). `AskInput` requires question (10-500 chars). Invalid inputs are rejected immediately with clear error messages.

Lifespan Management

ChromaDB connection is initialized once at server startup via an async context manager and reused across all tool calls. This avoids reconnecting on every request.

9. Streamlit Web Interface

The Streamlit app (`src/ui/app.py`) provides a web-based interface for document upload and interactive Q&A. It is intentionally thin - all logic is delegated to pipeline modules.

Layout

- Left column: document status indicator, file upload, processing results
- Right column: chat interface with message history, answer display with expandable sources

Key Features

- Real-time document status: shows indexed chunk count, warns if no documents loaded
- PDF upload: saves BytesIO to temp file (PyMuPDF needs real path), runs full pipeline
- Processing feedback: displays page count, chunk count, avg chunk size after ingestion
- Chat interface: maintains message history via `st.session_state` across reruns
- Source display: expandable panel showing citations, relevance scores, and chunk previews

10. Testing Strategy

The project has 33 unit tests across 4 test modules, all designed to be deterministic, fast, and cost-free. No real API calls are made during testing.

Testing Philosophy

- 100% mocking of external services (OpenAI API, ChromaDB) - no network calls
- Tests verify interface contracts: what we call, what we expect back
- Each pipeline module is tested in isolation
- Fixtures provide realistic sample data without touching real PDFs or APIs

Test Modules

`test_pdf_reader.py` - Data Models & I/O

- PageContent construction and type validation
- DocumentContent: full text concatenation, page lookup, metadata
- `_clean_page_text`: whitespace handling, hyphenation fixing, edge cases
- `extract_text_from_pdf`: file not found, non-PDF rejection

`test_chunker.py` - Cleaning & Splitting

- `detect_repeating_lines`: footer detection, threshold sensitivity, empty documents
- `clean_document`: footer removal, content preservation, immutability verification
- `chunk_document`: chunk production, metadata correctness, size limits, citation format
- `get_chunk_stats`: empty list handling, correct statistics calculation

`test_vector_store.py` - Vector Operations

- `embed_chunks`: correct upsert calls, metadata inclusion, empty list handling
- `query_collection`: result structure, distance calculation, empty collection
- Uses `@patch` to replace ChromaDB client and collection with MagicMock

`test_generator.py` - Answer Generation

- `build_user_prompt`: question inclusion, chunk text, citations, ordering
- `generate_answer`: return type, `has_answer` detection, source mapping, empty retrieval
- Answer model: source deduplication in formatted output

Running Tests

```
# All tests
pytest tests/ -v

# With coverage
pytest tests/ -v --cov=src --cov-report=term-missing

# Result: 33 passed in ~13s
```

11. Evaluation Framework (RAGAS)

Why This Matters

Any engineer can build a RAG system. Few can evaluate one rigorously. Showing up with metrics like 'faithfulness: 0.73, answer relevancy: 0.86' signals production engineering mindset, not just prototype thinking.

Three RAGAS Metrics

Faithfulness (0.0 - 1.0)

Does the answer stay within retrieved context? Detects hallucinations - claims the LLM made that are not supported by the provided chunks. Computed by an LLM judge that checks each statement in the answer against the context.

Answer Relevancy (0.0 - 1.0)

Does the answer address the actual question? Detects off-topic responses. The LLM judge generates questions from the answer and measures how similar they are to the original question using embeddings.

Context Precision (0.0 - 1.0)

Are the retrieved chunks actually relevant to the question? Measures retrieval quality. Low scores indicate the vector search is returning noisy, irrelevant chunks.

Evaluation Dataset (evaluation/dataset.py)

8 hand-crafted question/ground-truth pairs across categories: definition, classification, and process questions about MISRA compliance. Hand-crafting (vs auto-generation) demonstrates domain understanding and evaluation rigor.

Evaluation Pipeline (evaluation/evaluate.py)

- Step 1: `collect_rag_outputs()` - run each question through full RAG pipeline, store question + answer + contexts + ground_truth
- Step 2: `run_ragas_evaluation()` - convert to HuggingFace Dataset format, score with RAGAS using LLM-as-judge (gpt-4o-mini)
- Step 3: `save_results()` - write timestamped JSON to `evaluation/results/`
- Step 4: `print_report()` - display scores with threshold warnings

Implementation Challenges & Solutions

The RAGAS evaluation pipeline required significant debugging due to breaking API changes across RAGAS versions. This section documents the issues encountered and solutions applied.

Challenge 1: RAGAS Metric Import Changes (v0.4.x)

RAGAS v0.4.x introduced a parallel metrics API (`ragas.metrics.collections`) alongside the legacy API

(ragas.metrics.base). The new 'collections' classes (Faithfulness, AnswerRelevancy, ContextPrecision) inherit from a different base class (BaseMetric) than what evaluate() expects (Metric from ragas.metrics.base).

Solution: Use the legacy singleton metrics from the private modules (ragas.metrics._faithfulness.faithfulness, etc.) which pass the isinstance(m, Metric) check that evaluate() performs. These are pre-instantiated instances, not classes to call.

Challenge 2: Environment Variable Propagation

RAGAS internally creates its own OpenAI() client for LLM-as-judge evaluation. The project uses pydantic-settings to read .env files, but pydantic-settings only populates its own Settings object - it does not set os.environ. RAGAS's OpenAI() client checks os.environ['OPENAI_API_KEY'] and failed with 'api_key must be set'.

Solution: Added load_dotenv() from python-dotenv at the top of evaluate.py to populate os.environ before any RAGAS code runs.

Challenge 3: Result Format Change

RAGAS v0.4.x changed the EvaluationResult format. Previously result['faithfulness'] returned a single float. Now it returns a list of per-question scores. The save_results() and print_report() functions called float(scores['faithfulness']), which failed with 'float() argument must be a string or real number, not list'.

Solution: Added a _mean_score() helper that checks the type and computes the average if the value is a list, or returns the float directly if it's already scalar.

Challenge 4: Explicit LLM/Embeddings Requirement

RAGAS v0.4.x deprecated auto-detection of the LLM client. The evaluate() function accepts optional llm and embeddings parameters. Without them, RAGAS tries to create OpenAI() internally. With them, you have full control over which model judges your system.

Solution: Explicitly created judge LLM and embeddings using LangchainLLMWrapper and LangchainEmbeddingsWrapper, passing them to evaluate(). This also allows using a different model for judging (gpt-4o-mini) than for generation.

Latest Evaluation Results

```
RAG EVALUATION REPORT
Questions evaluated:  8
Questions answered:   7

SCORES (0.0 - 1.0, higher is better):
Faithfulness:        0.726
Answer Relevancy:    0.864
Context Precision:    0.312

ISSUES DETECTED:
Low context precision - retrieval returning noisy chunks
```

The low context precision (0.312) indicates the retrieval step is returning chunks that aren't highly relevant to the questions. Potential improvements: tune chunk size/overlap, use a better embedding model, add metadata filtering, or implement hybrid search (vector similarity + keyword BM25).

12. Containerization (Docker)

Two-Stage Build Strategy

The Dockerfile uses a two-stage build to minimize the final image size. Stage 1 (builder) installs all dependencies using uv. Stage 2 (runtime) copies only the installed packages and application source code.

```
Stage 1 (builder):
FROM python:3.11-slim
Install uv, copy pyproject.toml
uv pip install --system --no-cache '.[eval]'

Stage 2 (runtime):
FROM python:3.11-slim
Copy site-packages from builder
Copy src/, evaluation/, pyproject.toml
pip install --no-deps -e . (register package)
Configure Streamlit, expose port 8501
HEALTHCHECK every 30s
```

docker-compose.yml

- Port mapping: 8501:8501 (Streamlit)
- Volume mounts: ./data:/app/data and ./vectorstore:/app/vectorstore for persistence
- Environment: API keys from .env, configurable LLM provider and model
- Restart policy: unless-stopped (auto-recover from crashes)

Key Docker Decisions

- Slim base image (python:3.11-slim): ~200MB vs ~1GB for full image
- uv for dependency installation: 10-100x faster than pip in build stage
- Volume mounts for data: PDFs and vectors persist across container restarts
- Health check: Docker restarts container if Streamlit stops responding

13. CI/CD Pipeline

GitHub Actions runs on every push and PR to main, enforcing code quality before merge.

Three Automated Checks

1. Lint with Ruff

Checks for style issues, import ordering, naming violations, and common bugs. Also verifies consistent formatting.

2. Type Check with MyPy

Static type checking catches type mismatches at build time rather than runtime.

3. Run Tests with pytest

All 33 tests must pass. Generates coverage report. No code reaches main without passing all checks.

14. Technologies Summary

Technology	Version	Purpose
PyMuPDF	1.24+	PDF text extraction (C-based, fast)
LangChain	0.3+	Text splitter abstractions
OpenAI API	GPT-4o-mini	LLM generation + embeddings
Anthropic API	Claude	Alternative LLM provider
ChromaDB	0.5+	Local vector database
Pydantic	2.0+	Data validation + settings
Streamlit	1.38+	Web UI framework
FastMCP	1.0+	MCP server (Claude Desktop)
RAGAS	0.4+	RAG evaluation metrics
pytest	8.0+	Unit testing framework
Ruff	0.6+	Linter + formatter
MyPy	1.11+	Static type checking
Docker	24+	Containerization
GitHub Actions	-	CI/CD automation
uv	latest	Fast dependency management

15. Interview Talking Points

This section provides structured answers for common interview questions about the project. Each subsection covers a key topic area with concrete talking points you can use to demonstrate depth of understanding.

RAG Architecture

Q: Walk me through your RAG pipeline end-to-end.

The pipeline has five stages. First, Ingestion: PyMuPDF extracts text from PDFs page-by-page, preserving page numbers for citations. Second, Cleaning: a frequency-analysis algorithm detects repeating headers and footers by scanning which lines appear on 40%+ of pages, then strips them. Third, Chunking: LangChain's RecursiveCharacterTextSplitter breaks text into ~1000-character segments with 200-character overlap, trying paragraph breaks first, then sentence boundaries, then word boundaries. Fourth, Embedding: OpenAI's text-embedding-3-small converts each chunk into a 1536-dimensional vector stored in ChromaDB. Fifth, Generation: given a user query, we embed it with the same model, retrieve the top-5 most similar chunks via cosine similarity, inject them into a structured prompt, and call GPT-4o-mini (or Claude) at temperature=0 to produce a cited answer.

Q: Why recursive character splitting instead of fixed-size?

Fixed-size splitting cuts mid-sentence or even mid-word, destroying semantic context. For example, 'Rule 8.4 states that a compatible decla-' is useless as a chunk. RecursiveCharacterTextSplitter tries separators in priority order: paragraph breaks, newlines, sentence boundaries, word boundaries. This means chunks align with natural language structure. The result is chunks that are semantically coherent, which directly improves both retrieval accuracy and generation quality because the LLM receives complete thoughts, not fragments.

Q: What does chunk overlap do and why 200 characters?

Overlap creates a sliding window so that if important context spans a chunk boundary, both adjacent chunks contain it. Without overlap, a question about a concept that starts at the end of chunk N and continues at the start of chunk N+1 would miss the full context. 200 characters (roughly 1-2 sentences) is enough to capture boundary context without excessive duplication. This is about 20% of our 1000-char chunk size, which is a standard ratio in production RAG systems.

Q: How does your header/footer detection work?

It uses content-based frequency analysis rather than coordinate-based detection. The algorithm scans every page, counts how many pages each unique line appears on, and flags any line that appears on 40%+ of pages AND is under 200 characters. The character limit prevents removing long content lines that happen to repeat. This approach is self-adapting - it learns the pattern from the document itself, so it works on any PDF layout without manual configuration. It handles multi-line headers naturally and the threshold is configurable.

Vector Search & Embeddings

Q: How do embeddings work in your system?

Embeddings convert text into points in a high-dimensional geometric space where semantic similarity maps to spatial proximity. The sentence 'What is a deviation?' and the chunk 'A deviation is a documented permission to violate a guideline' end up as nearby vectors even though they share few exact words. We use OpenAI's text-embedding-3-small which produces 1536-dimensional vectors. Crucially, we use the same embedding model for both document chunks and user queries - this ensures they live in the same vector space. Using different models would be like measuring one distance in miles and another in kilometers, then comparing the numbers directly.

Q: Explain cosine similarity and why you use it.

Cosine similarity measures the angle between two vectors, ignoring magnitude. A score of 1.0 means identical direction (semantically identical), 0.0 means orthogonal (unrelated), and -1.0 means opposite. We use it instead of Euclidean distance because cosine similarity is magnitude-invariant - a short chunk and a long chunk about the same topic will have similar direction even if their vector magnitudes differ. ChromaDB returns distance (1 - cosine similarity), so lower distance means more similar. In the UI, we display relevance as (1 - distance) to show it as a percentage where higher is better.

Q: How does top-k retrieval balance precision and recall?

We retrieve $k=5$ chunks per query. This is a deliberate tradeoff. Too few ($k=1-2$) gives high precision but risks missing relevant context spread across multiple chunks. Too many ($k=10-20$) improves recall but floods the LLM context with noise, increasing hallucination risk and token costs. At $k=5$, we capture enough context for multi-faceted answers while keeping the noise manageable. Our RAGAS context precision score of 0.31 actually suggests we could benefit from reducing k or adding a re-ranking step to filter out the less relevant chunks before they reach the LLM.

LLM Integration & Hallucination Prevention

Q: How do you prevent the LLM from hallucinating?

We have five defensive layers. First, the system prompt explicitly forbids using knowledge outside the provided context - the LLM is instructed to only cite what it can find in the retrieved chunks. Second, we feed only the top-5 retrieved chunks to the LLM, not the full document - this limits the scope of what it can reference. Third, the prompt requires inline citations after every claim in the format '[Source: filename, page X]', which forces the LLM to ground each statement. Fourth, we detect the phrase 'does not contain information about this topic' in the response and set a `has_answer=False` flag - an honest 'I don't know' is better than a confident hallucination. Fifth, we run RAGAS faithfulness evaluation which quantitatively measures what percentage of claims in the answer are supported by the retrieved context. Our score of 0.73 means 73% of statements are fully grounded.

Q: Why temperature=0?

Temperature controls the randomness of LLM output. At `temperature=0`, the model always picks the highest-probability next token, making responses deterministic and reproducible. The same question with the same context will always produce the same answer. This is essential for a technical Q&A system because we want factual, consistent answers - not creative variations. If a user asks the same question twice, they should get the same answer. Higher temperatures introduce randomness that could lead to different phrasings, different emphasis, or even different factual claims.

Q: How does your provider abstraction work?

The `generate_answer()` function is the only public interface. Internally, `_call_llm()` dispatches to either `_call_openai()` or `_call_anthropic()` based on a single config setting (`settings.llm_provider`). Switching from GPT-4o-mini to Claude requires only changing two environment variables: `LLM_PROVIDER=anthropic` and `LLM_MODEL=claude-3-5-sonnet`. No code changes needed. This is like a HAL (Hardware Abstraction Layer) in embedded systems - the caller does not know or care which LLM provider is being used underneath. The abstraction also makes it easy to add new providers (e.g., Gemini, Llama) by implementing a single `_call_newprovider()` function.

Testing & Evaluation

Q: Describe your testing strategy.

We have 33 unit tests across 4 modules that run in ~13 seconds with zero API costs. Every external service (OpenAI API, ChromaDB) is fully mocked using `unittest.mock.patch`. The tests verify interface contracts: what we call, with what parameters, and what we expect back. For example, `test_generate_answer` patches `query_collection` to return known chunks and patches `_call_llm` to return a known response, then verifies the `Answer` object has correct sources, `has_answer` flag, and answer text. This approach is deterministic (no flaky tests from network issues), fast (no API latency), and free (no token costs). Each pipeline module is tested in isolation with fixtures providing realistic sample data.

Q: Explain your RAGAS evaluation results.

We run three RAGAS metrics using an LLM-as-judge approach (gpt-4o-mini evaluates our system's outputs). Faithfulness scored 0.73, meaning 73% of claims in our answers are fully supported by retrieved context - good but room for improvement on grounding. Answer relevancy scored 0.86, meaning our answers directly address the questions asked - this is strong. Context precision scored 0.31, which is our weakest metric - it means only 31% of retrieved chunks are actually relevant to the question. This tells us the retrieval step is the bottleneck. The LLM is doing well with what it gets, but we're giving it too much noise. Our evaluation dataset has 8 hand-crafted questions across definition, classification, and process categories - hand-crafting rather than auto-generating demonstrates domain understanding.

Q: What would you improve based on these scores?

The 0.31 context precision is the clear priority. Several approaches would help: First, add a cross-encoder re-ranker between retrieval and generation. The bi-encoder (embedding model) is fast but approximate; a cross-encoder scores query-chunk pairs more accurately. Retrieve 20 chunks, re-rank, keep top 5. Second, implement hybrid search combining vector similarity with keyword BM25. Some technical terms (like 'MISRA Rule 8.4') are better matched by exact keywords than semantic similarity. Third, tune chunk size - our 1000-char chunks might be too large, causing noise within each chunk. Smaller chunks (500-600 chars) with more overlap could improve precision. Fourth, add metadata filtering so retrieval can target specific sections or documents.

Production Readiness

Q: What makes this production-ready vs. a prototype?

Several engineering practices distinguish this from a quick demo. Type safety: Pydantic models validate data at every pipeline boundary - PageContent, DocumentContent, Chunk, Answer, RetrievedContext all have defined schemas with runtime validation. A malformed chunk cannot silently propagate. Configuration: all settings are centralized in a single Pydantic Settings class with environment variable support. No hardcoded secrets, no magic strings, clear priority hierarchy (env vars > .env > defaults). Observability: comprehensive logging at every pipeline stage with structured messages showing what's happening (e.g., 'Embedding 107 chunks...', 'Retrieved 5 chunks for query...'). Containerization: Docker with two-stage build, health checks, volume mounts for data persistence, and docker-compose for one-command deployment. CI/CD: every push runs Ruff (linting), MyPy (type checking), and pytest (33 tests). Code cannot reach main without passing all three gates.

Q: How do you handle configuration and secrets?

All configuration flows through src/config.py which uses Pydantic BaseSettings. API keys come from a .env file (never committed to git - listed in .gitignore). The Settings class provides typed defaults for non-sensitive values (chunk_size=1000, llm_model='gpt-4o-mini') and requires explicit values for sensitive ones (api keys). In Docker, keys are passed through docker-compose environment section which reads from the host's .env file. A single global 'settings' instance is created on module import and shared across the application - Singleton pattern. This means changing any setting requires only editing .env or setting an environment variable, never touching code.

Q: Explain your Docker setup.

We use a two-stage build. Stage 1 (builder) installs all dependencies using uv (10-100x faster than pip). Stage 2 (runtime) copies only the installed packages and source code, keeping the image small (~200MB vs ~1GB with full build tools). Key decisions: python:3.11-slim base for minimal attack surface, volume mounts for data/ and vectorstore/ so PDFs and vectors persist across container restarts, a health check that pings Streamlit's health endpoint every 30 seconds so Docker auto-restarts unhealthy containers, and 'restart: unless-stopped' policy for crash recovery. The package is installed with 'pip install --no-deps -e .' in the runtime stage to register import paths without re-downloading dependencies.

Potential Improvements You Could Discuss

These are improvements you can mention proactively in interviews to show you understand the system's limitations and know what 'next level' looks like.

Re-ranking with Cross-Encoders

Current approach uses a bi-encoder (embedding model) for retrieval, which is fast but scores query and document independently. A cross-encoder (like ms-marco-MiniLM) scores the query-document pair jointly, producing more accurate relevance scores. The pattern is: retrieve 20 chunks with the fast bi-encoder, re-rank with the accurate cross-encoder, keep top 5. This directly addresses our low context precision (0.31) without changing the embedding model or vector database.

Hybrid Search (Vector + BM25)

Semantic search excels at meaning-based matching but struggles with exact terms. If a user asks about 'MISRA Rule 8.4', keyword matching (BM25) finds exact matches while vector search might return chunks about Rule 8.3 or 8.5 that are semantically similar. Hybrid search combines both: run vector search and BM25 in parallel, merge results using Reciprocal Rank Fusion (RRF). This gives the best of both worlds - semantic understanding plus keyword precision.

Streaming LLM Responses

Currently the UI waits for the complete LLM response before displaying anything. Streaming sends tokens back as they're generated, reducing perceived latency from 5-10 seconds to near-instant. Both OpenAI and Anthropic support streaming natively. Implementation requires changing the API call to use `stream=True` and yielding tokens to the Streamlit UI via `st.write_stream()` or a placeholder that updates incrementally. Note: we have already implemented this in `generate_answer_streaming()`.

Query Expansion

A single query may miss relevant chunks phrased differently. Query expansion uses the LLM to generate 3-5 variations of the original question, retrieves chunks for each variation, and merges the results. For example, 'What is a deviation?' expands to 'How are deviations defined?', 'What constitutes a MISRA deviation?', 'Define deviation in compliance context'. This casts a wider net during retrieval without changing the embedding model.

Multi-tenancy

Currently all documents go into a single ChromaDB collection. For multi-user or multi-project use, each tenant should have an isolated collection. ChromaDB supports multiple named collections natively. Implementation: add a 'project_id' to the Settings, use it as the collection name, and ensure queries only search within the user's collection. This prevents information leakage between tenants and allows per-project tuning of chunk size, overlap, and retrieval parameters.